AIMLCZG546 · SESSION 1 · COMPANION READER

Foundations of ML Systems Engineering

SDLC, Data Science, ML Pipeline, Three Generations, Domains and LLMs

Session 1 · Read alongside the lecture slides · Every concept explained from scratch

How to use this companion. Five colour-coded boxes appear throughout. **Blue** boxes give the core intuition in plain words. **Amber** boxes anchor the idea to an everyday situation you already understand. **Green** boxes walk through a concrete example step by step. **Red** boxes flag common pitfalls. **Purple** boxes state the single most important line to remember.

1 Software Development Life Cycle (SDLC)

Every software product you use — a banking app, a ride-hailing service, a hospital record system — was built using a repeatable process. That process is the **Software Development Life Cycle (SDLC)**: a structured sequence of phases that guides a team from an idea to a shipped product and then keeps it running.

The intuition

The SDLC is a recipe. A recipe tells a chef what to do, in what order, and how to know when each step is done. The SDLC tells a software team: first understand what to build, then design it, then write the code, then test it, then ship it, then maintain it.

★ Everyday picture

Think of building a house. Before a single brick is laid, the owner explains their needs (requirements). An architect draws blueprints (design). Builders construct the walls (implementation). Inspectors check for cracks (testing). The family moves in (deployment). A plumber fixes leaks over the years (maintenance). Software teams do exactly the same six things, in the same order.

The six classic SDLC phases are:

1. **Planning** — what are we building, and is it feasible?
2. **Requirements Analysis** — what exactly must the system do?
3. **System Design** — how will it be structured (architecture, database, APIs)?
4. **Implementation (Coding)** — write the actual code.
5. **Testing & Quality Assurance** — does it work correctly and safely?
6. **Deployment & Maintenance** — release it and keep it healthy.

1.1 Roles in a Software Team

A software project involves many specialists, each responsible for a different slice of the process.

Role	Phase focus	Core responsibility
Business Analyst	Requirements	Translates business needs into technical specifications
Project Manager	All phases	Plans timelines, manages risks, tracks progress
Product Owner	Requirements + Delivery	Represents users; prioritises features
Software Architect	Design	Designs high-level structure and technology choices
UX/UI Designer	Design + Testing	Creates user interfaces and experience flows
Developer	Implementation	Writes and reviews code
QA / Tester	Testing	Writes and runs tests; reports defects
Scrum Master	All (in Agile)	Removes blockers; facilitates Agile ceremonies
Team Lead	Implementation	Guides the developer team; does code review

Worked example: Who does what on a food-delivery app

Imagine building a Swiggy-like app. The *Business Analyst* interviews restaurant owners and hungry customers to write a requirements document. The *Software Architect* decides to use microservices (separate services for orders, payments, and delivery tracking). The *UX Designer* draws the app screens. *Developers* build each service. *QA engineers* test that placing an order deducts the right amount and notifies the right delivery partner. The *Project Manager* ensures the first release ships before the next festive season. The *DevOps engineer* deploys the app to the cloud and sets up monitoring.

1.2 Evolution to Cloud-Native

Traditional SDLC produced software released once every six to twelve months. Modern software teams ship multiple times per day. The shift happened through three combined innovations:

- **Agile** — work in short sprints (1–3 weeks); deliver working software continuously.
- **DevOps** — development and operations teams share responsibility; automate testing and deployment.
- **Cloud Native** — use containers (Docker), orchestration (Kubernetes), microservices, and public cloud infrastructure to build systems that are small, independent, and always running.

Where this shows up in ML/AI. Every ML team operates within some variant of SDLC. The critical insight is that ML adds new phases (data collection, model training, model evaluation) that don't appear in traditional SDLC. The rest of this course is about adapting the SDLC to accommodate that.

✓ Key takeaway

The SDLC is the backbone of every software project. It doesn't change when ML enters the picture — ML adds new phases inside it.

2 Data Science and the AI/ML Landscape

Data Science is the study of data: collecting it, cleaning it, analysing it, and drawing actionable insights from it. The goal is to extract value that would otherwise remain hidden inside raw numbers.

The intuition

Imagine a mountain of sand. Data Science is the sieve that separates the gold flakes (insights) from the rest (noise). The sieve has five mesh layers: collect the sand, clean it, analyse its composition, build a model to predict where more gold is, and visualise the results for others to act on.

★ Everyday picture

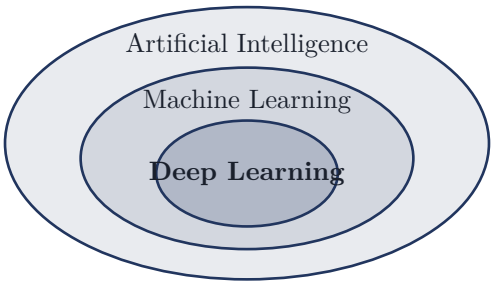
A doctor who reviews thousands of patient records to notice "patients with these three symptoms are 80 % more likely to develop diabetes within two years" is doing data science. They collected data (records), cleaned it (removed duplicates), analysed it (spotted the pattern), built a prediction model in their head, and communicated the finding (the 80 % figure) to their colleagues.

The standard Data Science workflow:

1. **Data Collection** — gather raw data from databases, APIs, sensors, surveys.
2. **Data Preprocessing** — fix missing values, remove duplicates, normalise formats.
3. **Exploratory Data Analysis (EDA)** — visualise distributions, spot correlations, understand the data.
4. **Modelling** — train a statistical or ML model on the cleaned data.
5. **Deployment & Visualisation** — share the predictions or dashboards with decision-makers.

2.1 Where AI, ML, and Data Science Overlap

These three terms are often confused. Here is the precise relationship:



- **Artificial Intelligence (AI)** — the broad goal of making machines behave intelligently.
- **Machine Learning (ML)** — a subset of AI where machines learn from data rather than being hand-programmed with rules.
- **Deep Learning** — a subset of ML that uses multi-layer neural networks; excels when data is large and unstructured (images, text, audio).
- **Data Science** — overlaps all three; focuses on extracting insights from data, often using ML as a tool.

Where this shows up in ML/AI. Data Science is often the starting point of an ML project. Before a model can be trained, a data scientist must collect and clean the data. The roles overlap significantly; in small companies one person may do all of them.

3 Machine Learning Basics

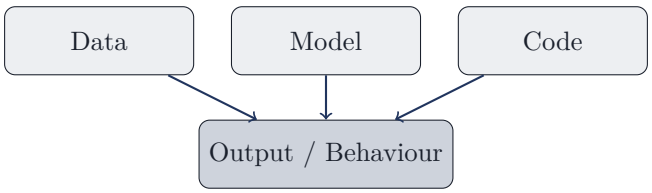
Machine Learning is a branch of AI in which a computer program improves its performance on a task through experience (data), without being explicitly programmed with rules for that task.

The intuition

Traditional programming is like giving someone a recipe: "If the tomato is red and soft, it is ripe." ML is like showing that person a thousand tomatoes labelled "ripe" or "not ripe" and letting them figure out the rule themselves. The rule they learn may be far more nuanced than anything you could write down.

3.1 The Three-Variable Complexity of ML

Traditional software has one moving part: the *code*. You change the code, the behaviour changes. ML systems have three moving parts that all affect the output:



Change the data (add more examples or remove biased ones) and the behaviour changes. Change the model architecture (switch from logistic regression to a neural network) and the behaviour changes. Change the code (bug in the preprocessing pipeline) and the behaviour changes. This three-way dependency makes debugging ML systems much harder than debugging traditional software.

✗ Watch out

Do not test an ML system by only testing the code. A bug in the training data (mislabelled examples, wrong units) will silently produce wrong predictions even when all the code is correct. You must test data quality, model behaviour, and code together.

3.2 The Model Lifecycle

A **model** is a trained algorithm that accepts inputs and produces predictions or decisions. A model is not static — it has its own lifecycle:

1. **Development** — choose an algorithm, train it on labelled data, tune hyperparameters.
2. **Evaluation** — measure accuracy, precision, recall, AUC on a held-out test set.
3. **Versioning** — save the trained model file and all metadata (training date, dataset version, metrics).
4. **Deployment** — serve the model via an API so applications can call it.
5. **Monitoring** — track prediction quality in production; detect data drift and performance degradation.
6. **Retraining** — retrain on fresh data when performance drops below an acceptable threshold.

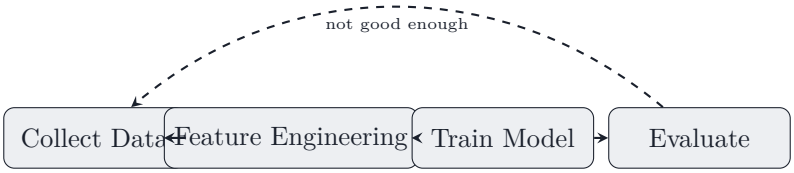
Where this shows up in ML/AI. Every MLOps tool (MLflow, SageMaker, Vertex AI) is

built around this lifecycle. When you hear "ML pipeline" in industry, this is what people mean.

4 The ML Pipeline

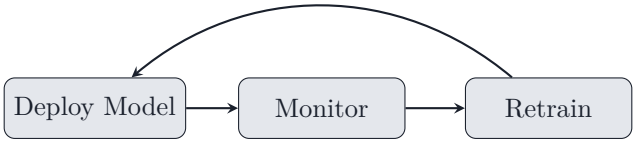
The **ML pipeline** is the end-to-end sequence of steps that takes raw data and produces a deployed, monitored model. It has two linked loops.

4.1 The Static (Training) Pipeline



4.2 The Production Pipeline

After the model passes evaluation, it enters production:



Worked example: Spam filter pipeline

Collect: gather 100,000 emails labelled "spam" or "not spam".
Features: extract word counts, sender domain reputation, link count.
Train: fit a logistic regression model on 80,000 training emails.
Evaluate: test on 20,000 held-out emails — precision 94 %, recall 91 %.
Deploy: expose the model as a REST API that email servers call for each incoming message.
Monitor: track the fraction of emails users manually move to/from spam. If it rises above 10 %, the model is drifting.
Retrain: re-run the pipeline on the latest 30 days of emails every week.

5 Three Generations of Machine Learning

ML has evolved through three distinct generations, each solving problems the previous generation could not.

Generation	Era	Key technique	What you need
Gen 1: Classic ML	1990s–2010s	Decision trees, SVMs, linear models	Labelled datasets, feature engineering
Gen 2: Deep Learning	2012–2020	Neural networks (CNN, RNN, LSTM)	Big data, GPU compute, labelled data
Gen 3: Transfer / Foundation	2020–now	Pre-trained models, fine-tuning, APIs	A task description; no training required

★ **Everyday picture**

Gen 1 is like hiring an apprentice who learned only what you explicitly taught them. Gen 2 is like hiring a savant who learned by reading millions of books — but you still need to show them the books. Gen 3 is like using a consultant who already knows everything; you

just tell them what you need and they deliver.

✓ Key takeaway

Gen 3 (Foundation Models) dramatically lowers the barrier to entry: a developer can add ML capability to an app by calling an API, with no training data at all.

6 Types of ML Domains

ML is applied across many domains. The three most prevalent in industry are:

6.1 Natural Language Processing (NLP)

NLP teaches machines to understand, generate, and reason about human language. Human language is ambiguous, context-dependent, and culturally loaded — making rule-based systems inadequate.

Examples: sentiment analysis, machine translation (Google Translate), document summarisation, question answering, chatbots.

6.2 Computer Vision (CV)

Computer Vision gives machines the ability to interpret visual data (images, video). Applications: object detection in autonomous vehicles, face recognition for unlocking phones, quality control in manufacturing (spotting defects on a production line), medical imaging (detecting tumours in X-rays).

6.3 Speech Recognition

Automatic Speech Recognition (ASR) converts spoken audio to text. Used in virtual assistants (Alexa, Siri, Google Assistant), transcription services, call-centre automation.

Where this shows up in ML/AI. Most modern ML applications combine domains. A voice assistant uses ASR (speech → text) + NLP (understand the question) + text-to-speech (answer). An autonomous car uses CV (see the road) + NLP (process voice commands) + reinforcement learning (decide how to steer).

7 Foundation Models and Large Language Models

A **Foundation Model** is a large-scale model trained on a massive, broad dataset (the entire web, all of Wikipedia, billions of images) and then adapted to specific tasks. The training is expensive; the adaptation is cheap.

The intuition

Think of a foundation model as a university-educated generalist. They spent years learning widely (pre-training). You then give them a two-week onboarding (fine-tuning) for your specific job and they are immediately productive. Contrast this with a Gen 1 specialist who only learned what you explicitly taught them — they are only as good as the examples you provided.

Large Language Models (LLMs) are the dominant type of foundation model today. Trained on text, they can write, summarise, translate, answer questions, generate code, and reason

step by step. Examples: GPT-4 (OpenAI), Claude 3.5 (Anthropic), Gemini (Google), LLaMA (Meta), DeepSeek, Mistral.

7.1 What an AI Engineer Needs to Know

The modern **AI Engineer** role bridges software engineering and AI. Required skills:

- Software engineering fundamentals: Python, SQL, Git, CI/CD, REST APIs.
- LLM integration: prompt engineering, RAG (Retrieval-Augmented Generation), fine-tuning.
- Model Context Protocol (MCP): connecting LLMs to external tools.
- MLOps: versioning, monitoring, deployment pipelines.

8 Software Engineering vs Machine Learning Engineering

SE and ML engineering are siblings, not twins. Understanding their differences is the first step to applying SE skills effectively in ML projects.

Dimension	Traditional SE	ML Engineering
Specification	Clear, explicit (requirements doc)	Fuzzy — defined by data and metrics
Correctness	Deterministic: same input → same output	Probabilistic: output depends on model and data
Debugging	Trace code; add print statements	Inspect data; plot distributions; ablation studies
Testing	Unit tests with expected outputs	Test on held-out data; measure F1, AUC
Artefacts	Executable code	Executable code + trained model + dataset
Maintenance	Fix bugs, add features	Retrain; fix data pipelines; monitor drift
Failure mode	Exception thrown; stack trace	Silently wrong predictions

✗ Watch out

The most dangerous ML failure is a *silent* one. A software bug usually crashes the program and announces itself. An ML model that starts making wrong predictions may keep running and serving wrong answers for days before anyone notices. Monitoring is not optional.

Worked example: Where the difference hurts in practice

A payment gateway is built with traditional SE. When the fraud-detection rule "decline if country mismatches" fires incorrectly, the team adds an exception and redeploys in hours. Now replace the rule with an ML fraud model. The model starts declining legitimate transactions from a new country market because it was never trained on them. The only symptom is a slow increase in customer complaints. Finding and fixing this requires: inspecting model predictions on the new market, checking whether the training set included that market, retraining the model, evaluating on a fresh test set, and redeploying — a multi-day engineering effort. This is why SE skills adapted to ML are essential.

✓ Key takeaway

ML shifts the source of truth from code to data. That single fact changes how you specify, test, debug, and maintain a system.